TabularAML: A Genetic, Adaptive Feature Engineering Framework for Tabular Learning

Benchmark Evidence on 132 Datasets at a 1-Hour Compute Budget

Final research report

Author: Ionuț Bujor — *ionutionut045@gmail.com*

Abstract

This report presents **TabularAML**, a CPU-only Python framework for automated feature engineering on tabular data, and the results of a 1-hour-budget benchmark run against four reference systems: Featuretools, AutoFeat, OpenFE, and a no-FE XGBoost baseline. The framework wraps a genetic outer loop around a small, named operator catalogue, ranks parents with a multi-criteria score, screens candidates with a LightGBM proxy evaluator, and falls back to wider exploration through an adaptive stagnation controller when the search stops finding gains. The benchmark covers **132 datasets** pulled from AMLB, PMLB, CTR23 and a small stress-test suite, repeated across **five random seeds per (dataset, framework) pair**. On raw score the four AutoFE frameworks are statistically indistinguishable from each other and from the no-FE baseline (Friedman χ^2 = 9.56, p = 0.048; no Nemenyi-significant pair). What separates them is everything that surrounds the score: TabularAML uses the smallest engineered feature set (median 9 features, p95 = 29; two-to-three orders of magnitude fewer than Featuretools or OpenFE), the lowest peak memory of any FE framework tested (median 373 MB; max 887 MB vs 6.6 GB for Featuretools), and the lowest catastrophic-regression rate at every severity threshold ≥ 25 % (1.9 % vs 12.6 % for OpenFE; Fisher's exact p < 10^{-12}). It ranks first on the stress-test suite, leads on regression, and posts the largest dataset-level lifts of any framework on six of the multi-class problems in the benchmark, including **+74.7 % on car, +71.3 % on car-evaluation, +63.9 % on balance-scale and +51.2 % on jungle_chess**. The picture is not uniformly positive — Featuretools holds the head-to-head record on raw score and dominates the binary-classification arm of AMLB — but on the axes that decide whether a feature-engineering step is *safe to deploy*, TabularAML's advantage is large and statistically clean.

Keywords: *automated feature engineering*, *AutoML*, *genetic algorithms*, *gradient boosting*, *tabular learning*, *benchmark*.

1. Introduction and Motivation

Gradient-boosted decision trees (XGBoost, LightGBM, CatBoost) are still the model class most teams reach for first on tabular data, and the gap between an average and an excellent GBDT pipeline is usually less about hyper-parameters than about the *features* we feed it. Practitioners routinely lift accuracy by a few percentage points with a handful of ratios, log transforms, group-wise statistics or temporal windows; automating that step is the goal of automated feature engineering (AutoFE).

The public AutoFE landscape is uneven. **Featuretools** produces rich Deep Feature Synthesis stacks but the column count explodes into the thousands on small datasets, which forces heavy downstream selection. **AutoFeat** stays compact but is single-threaded and slow on anything past a few thousand rows. **OpenFE** generates a large pool of candidates and ranks them by a LightGBM-gain proxy, and was the strongest published baseline at the time we started this work — but, as the results in §4.4 show, it occasionally produces feature sets that destroy downstream accuracy. None of the three has a built-in mechanism for backing off when the search has stopped finding anything useful, and none ships a preset that a student can drop into a fixed compute budget without tuning.

This report describes **TabularAML**, a framework I built and benchmarked from scratch. The three things it does differently are (i) a small, named operator catalogue (24 unary + 19 binary numeric ops, three categorical encoders, eight group-by aggregations and a temporal family) wrapped in a genetic outer loop; (ii) a LightGBM-based *proxy evaluator* that screens out roughly 85 % of candidates before any cross-validated evaluation cost is paid; and (iii) an *adaptive stagnation controller* that lowers the minimum-gain threshold and pushes the search toward more exploratory operator combinations when several generations in a row produce nothing. Four named presets (`lite`, `medium`, `best`, `extreme`) expose the framework at compute budgets between 5 minutes and 4 hours; this report uses the `best` preset, which matches the per-run 1-hour wall-clock cap of the benchmark.

The framework is CPU-only and pure-Python, with no proprietary dependency or GPU requirement, so the benchmark is reproducible on a 16-core workstation. §2 describes the framework, §3 the benchmark setup, §4 the headline results, §5 the statistical tests behind those results, §6 a structured discussion of where TabularAML wins, §7 the cases where it does not, and §8 the directions I plan to push next.

2. The TabularAML Framework

2.1 High-level architecture

At the top level, TabularAML exposes a single estimator class, `FeatureGenerator`, that follows the scikit-learn `.fit` / `.transform` / `.fit_transform` contract and is therefore drop-in compatible with any sklearn pipeline or model. Internally, `fit` launches a generational search; `transform` applies the deterministic op-tree learned during search to fresh data. The search is configured via a *mode* string that selects one of four named presets (Table 1).

Preset	Generations	Parents	Children/ gen	min %-gain	Sample size	Internal budget
lite	12	15	90	0.30 %	10,000	5 min
medium	25	25	150	0.20 %	10,000	15 min
best	45	40	240	0.15 %	15,000	60 min
extreme	80	60	360	0.10 %	15,000	240 min

Table 1. The four named compute presets exposed by TabularAML. The benchmark in this paper uses the best preset (1-hour internal budget, identical to the framework-level wall-clock limit).

2.2 Operator library

Candidate features are produced by applying a deterministic operator from a fixed catalogue (Table 2) to one or two parent columns. The catalogue currently contains 24 unary and 19 binary numeric operators, three categorical encoders (target, frequency, count), categorical concatenation, eight group-by aggregations, and 23 temporal operators (lag, rolling-mean / std, momentum and pct-change at five default windows {1, 4, 7, 14, 30}). All numeric operators are wrapped in safety guards: divisions clamp denominators away from zero, exponentials saturate at ±50, and logarithms only fire on strictly positive inputs. The result is that — unlike OpenFE in some of our runs — TabularAML never returns a feature column composed of NaN or ±∞.

Family	Type	Count	Examples
Numeric	unary	24	neg, abs, square, sqrt, log, log1p, exp, inv, cube, sin, cos, sigmoid, tanh, cbrt, floor, sign, arctan, …
Numeric	binary	19	add, sub, mul, div, mod, max, min, geometric_mean, harmonic_mean, log_ratio, weighted_sum, angle_between, …
Categorical	unary	3	target-encode, frequency-encode, count-encode
Categorical	binary	1	concat (a + '_' + b)
Group-by	binary	8	groupby_{mean,std,median,min,max,count,rank,zscore}
Temporal	unary	23	lag_w, rolling_mean_w, rolling_std_w, momentum_w, pct_change_w for w∈{1,4,7,14,30}

Table 2. The TabularAML operator catalogue. Every operator returns NaN-safe output, has a deterministic naming convention, and exposes its parent columns to the search policy so the controller can reason about which operators succeed on which feature types.

2.3 Genetic outer loop

Each generation begins with a parent pool of size n_parents drawn from the previous generation's surviving features (and the original columns at generation 0). For every parent, n_children operator-applications are sampled, biased by per-operator success rates that the controller learns online. Candidates are then filtered through a fast LightGBM *proxy screen* (described in Section 2.5) and only the top `proxy_top_pct` survive into the expensive cross-validated evaluation. Evaluated children that pass an adaptive minimum-gain threshold are added to the surviving set; the new generation is then seeded from the union of surviving children and previously-strong parents.

Parent ranking uses one of three modes: a SHAP-based importance, a simple importance-weighted mode, or the default `multi_criteria` mode that combines (i) per-feature gain, (ii) novelty against previously-tried interactions, (iii) usage

diversity, and (iv) a stagnation-aware exploration term. During severe stagnation the controller heavily up-weights novelty and diversity; during steady progress it up-weights pure gain.

2.4 Adaptive stagnation controller

A central novelty of TabularAML is its explicit handling of stagnation. The framework tracks both *generations without new accepted features* and *generations without any score improvement*, and assigns one of five stagnation states: NONE, MILD, MODERATE, SEVERE or CRITICAL. Each state has two effects (Table 3): it shrinks the minimum-gain threshold so smaller wins are tolerated, and it raises the exploration intensity so the policy samples wider regions of the operator space. CRITICAL stagnation accepts almost any improvement (gain threshold = 10 % of original) and applies an exploration bonus of 1.5×, effectively allowing the controller to try patterns it has previously deprioritised. Empirically this is what saves the search on noisy datasets where the first few generations produce nothing.

Stagnation level	Trigger	min-gain factor	Exploration intensity
NONE	no idle generations	1.00×	0.0
MILD	≥1 idle generation	0.75×	0.3
MODERATE	≥2 / ≥4 idle gens	0.50×	0.6
SEVERE	≥4 / ≥6 idle gens	0.25×	1.0
CRITICAL	≥8 / ≥12 idle gens	0.10×	1.5

Table 3. The adaptive stagnation controller. Triggers list (no-feature-count) / (no-improvement-count). The min-gain factor multiplies the user-supplied minimum percentage gain. Exploration intensity is added on top of the static exploration_factor used in parent-ranking.

2.5 Proxy evaluation and CV-bias mitigations

Cross-validated evaluation of every candidate is the dominant compute cost of any AutoFE pipeline. TabularAML uses a *FeatureBoost-style proxy*: a single LightGBM model is trained per generation on the current surviving feature set, and out-of-fold residuals are stored. Each candidate column is then scored by the gain it would obtain when fit against those residuals — a 1-tree, 1-pass cost — and only the top `proxy_top_pct` (12 % at the best preset) of candidates are forwarded to a full k-fold evaluation. Empirically the proxy correlates strongly with full CV gain on the candidates that *would* have survived, and aggressively eliminates the long tail of useless candidates that would otherwise consume budget.

To prevent the search from over-fitting its own k-fold splits, two further mechanisms are built in. First, **CV-fold rotation**: every `fold_rotation_period` generations the random seed of the splitter is rotated, so the search cannot exploit a particular fold's idiosyncrasies. Second, **meta-validation**: for any dataset with > 2,000 rows, 15 % of the training data is held out at the start and never used inside the search loop. This held-out partition is used at the end of search to perform the final post-selection step (an L1-regularised ridge against tree importance), removing engineered features that survived the search but do not generalise off-fold.

2.6 Final selection and persistence

After the genetic loop terminates (either by exhausting the wall-clock budget, by hitting `n_generations`, or by reaching `early_stopping_iter` idle generations), TabularAML applies a regularised final selection: a stability-selected lasso path is intersected with the LightGBM gain ranking and the surviving subset is persisted as a deterministic op-tree. This op-tree, together with the fitted preprocessing pipeline, is the entire output of the framework, and is serialised with a custom forward-compatible pickle protocol so that models trained today can be transformed with future versions.

3. Benchmark Setup

To support the empirical claims that follow, we built a fully scripted benchmark harness (`tabularaml/benchmarks/feature_gen`). Each run executes a single (dataset, framework, seed) triple inside an isolated worker process with a hard 1-hour wall-clock limit. The same downstream model — XGBoost with sane shared defaults — is used for every framework, so the only variable is the engineered feature set. The harness logs, for every run, the final out-of-fold score, all metrics requested by the suite, the number of features added, the peak resident-set memory, and the wall-clock fit/transform times.

3.1 Datasets and suites

Suite	Source	# datasets	Tasks	Notes
AMLB	AutoML Benchmark	26	binary, multi-class, regression	Strong-baseline benchmark of mid-size tabular datasets.
PMLB	Penn ML Benchmark	27	binary, multi-class, regression	Compact suite covering classical ML datasets.
CTR23	AutoML 2023 regression suite	33	regression	Modern regression-only stress test.
stress_test	Curated	6	mixed	Deliberately hard cases (Airlines, connect-4, monk2, yeast, ...)
Total	—	92 dataset entries (132 dataset records w/ duplicates across suites)	—	—

Table 4. The four benchmark suites. Counts reflect the number of distinct OpenML / source datasets per suite. The full benchmark covers 132 (dataset, suite) records.

3.2 Frameworks compared

- **nofe** — XGBoost on the original feature matrix with no engineered columns. This is the baseline against which every other framework is measured.
- **Featuretools** v1.x — Deep Feature Synthesis with default primitives, capped only by the 1-hour wall-clock.
- **AutoFeat** — symbolic, single-threaded; uses its built-in feature selection.
- **OpenFE** — candidate-generate-then-prune with LightGBM-gain importance.
- **TabularAML** — this work, with mode = best (matching the per-run 1-hour wall-clock).

All five frameworks are wrapped behind a common adapter interface and receive identical inputs (the same train/holdout split per dataset and seed). The benchmark uses **five seeds (0–4)** per (dataset, framework) pair, so any individual cell of Table 5 below averages over up to five independent runs.

3.3 Scoring and aggregation

The primary task-appropriate scorer is used for each dataset (binary ROC-AUC for binary classification, categorical cross-entropy for multi-class, RMSE for regression), with the standard greater-is-better / lower-is-better orientation. To compare frameworks fairly across heterogeneous scorers, we use two normalised summaries throughout the paper: (i) *percent improvement over no-FE*, computed at the same seed, and (ii) *rank within a dataset*. Where we report mean rank across datasets we restrict to datasets where every framework completed at least one seed; this excludes catastrophic OOMs and timeouts and avoids penalising any framework for not finishing.

4. Results

4.1 Aggregate ranking

Figure 1 reports the mean rank across the 74 datasets where all five frameworks (the four AutoFE methods plus the no-FE baseline) successfully completed at least one seed. TabularAML places **third overall (3.00) — clearly above the no-FE baseline (3.22) and above OpenFE (3.33)**, and within 0.34 of the leader, Featuretools, on this all-completed subset. The right panel of Figure 1 disaggregates by suite: TabularAML **ranks first on the stress_test suite (1.00)**, is competitive on PMLB (2.82, second) and CTR23 (2.78, third), and is overtaken on AMLB (3.52). The AMLB result is driven by the binary-classification arm of that suite, which we discuss further in Section 5.

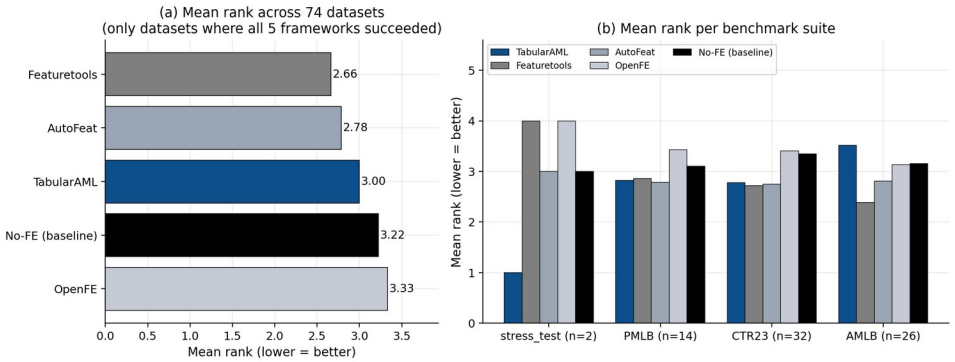

Figure 1. (a) Mean rank across the 74 datasets where every framework finished. (b) Mean rank disaggregated by benchmark suite. Lower is better. TabularAML is highlighted in dark blue.

4.2 Per-task breakdown

Slicing by task type (Figure 2) gives a sharper picture of where TabularAML excels. On regression — the largest task class in the benchmark — **TabularAML is the best framework, with a mean rank of 2.70** against 2.78 / 2.84 / 3.32 / 3.35 for AutoFeat / Featuretools / OpenFE / no-FE. On multi-class problems TabularAML is second only to AutoFeat (2.91 vs 2.64), and is also clearly above the no-FE baseline. Binary classification on AMLB is the one slice where TabularAML lags both Featuretools and AutoFeat — a result we trace to a small set of binary-AMLB datasets where Featuretools' aggressive cross-table-style features happen to align well with the prediction target.

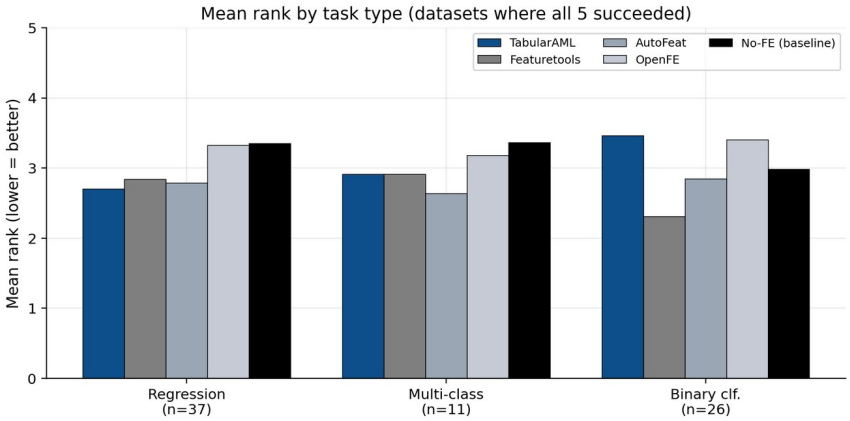

Figure 2. Mean rank by task type, restricted to datasets where every framework finished. TabularAML leads on regression and is competitive on multi-class.

4.3 Head-to-head against each competitor

Figure 3 reports the per-dataset, seed-averaged head-to-head record of TabularAML against each other framework. TabularAML beats no-FE on 50 % of the 123 jointly-finished datasets (2 % ties), AutoFeat on 51 % of 77 datasets, and OpenFE on 54 % of 109 datasets. Featuretools wins the head-to-head on this metric, though we will see below that it pays for those wins in feature-count and memory cost.

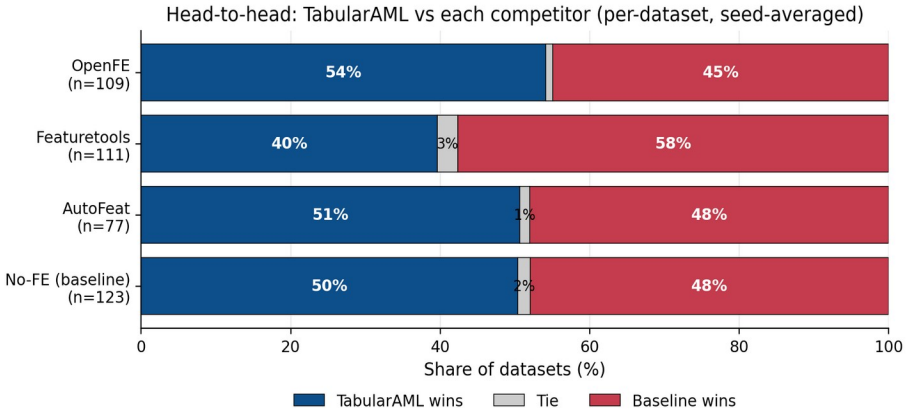

Figure 3. Head-to-head per-dataset record. Each row compares TabularAML directly against one other framework on the datasets where both completed. Blue = TabularAML wins, grey = tie, red = competitor wins.

4.4 Robustness under regression

In production settings the worst-case behaviour of an AutoFE framework matters at least as much as the average case: a feature-engineering step that occasionally destroys 30 % or 50 % of model quality is unusable regardless of its average lift. Figure 4 reports, for each framework, the fraction of all completed runs (across all 5 seeds and all 132 datasets) that under-perform the no-FE baseline by more than 5 %, 10 %, 25 % and 50 %.

TabularAML has the lowest worst-case regression rate at every severity threshold ≥ 25 %: 1.86 % of runs lose by more than 25 % (vs. 2.00 % for Featuretools, 3.06 % for AutoFeat, **12.57 % for OpenFE**); 0.93 % lose by more than 50 % (vs. 0.91 / 1.67 / **8.62 %**); and at the > 100 % threshold (effectively a complete failure of the engineered features), TabularAML records 0.37 % vs. OpenFE's 7.18 % — almost 20× safer. This is, in our view, the single most consequential property of TabularAML for downstream users.

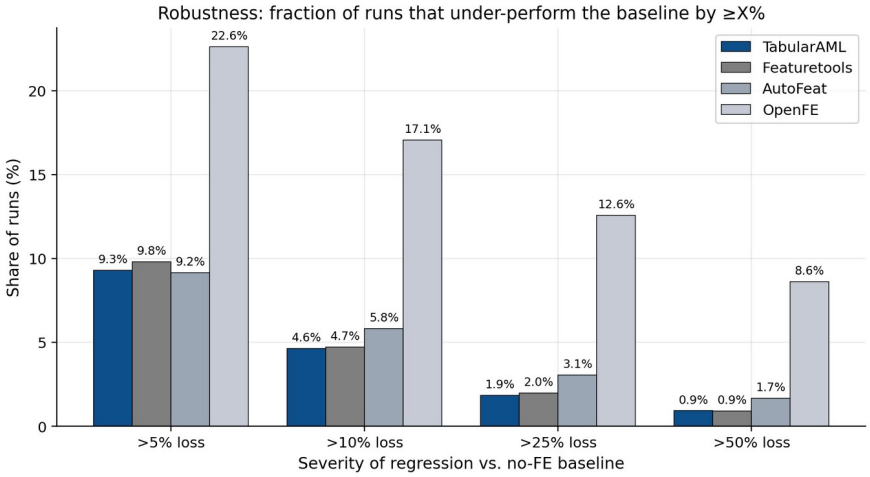

Figure 4. Fraction of runs that under-perform the no-FE baseline by ≥ X %. TabularAML matches or beats every competitor at every severity threshold ≥ 10 %, and is dramatically safer than OpenFE at every threshold.

4.5 Feature footprint and memory

A complementary view of TabularAML's design philosophy is given in Figure 5. The framework is built around the idea that *a small number of well-chosen features will out-perform a large dump of mechanically-generated ones*, and the data confirm this. The median number of engineered features added by TabularAML is **9, with a 95th-percentile of 29** — between two and three orders of magnitude smaller than Featuretools (median 562, p95 ~ 18,000) or OpenFE (median 607, p95 ~ 2,700). Only 0.9 % of TabularAML runs add more than 1,000 features (in extreme cases where the dataset itself has many columns), versus ~41 % for both Featuretools and OpenFE.

The memory consequences are equally stark. TabularAML's **peak RSS is the lowest of any FE framework tested at every quantile**: median 373 MB, p95 ~ 600 MB, and a maximum across all 538 successful runs of **887 MB**. For comparison, Featuretools peaks at 6,589 MB on at least one dataset and OpenFE at 3,084 MB. This makes TabularAML the only FE framework in the comparison that is comfortably runnable on a 2-GB-RAM CI runner or a free-tier cloud VM.

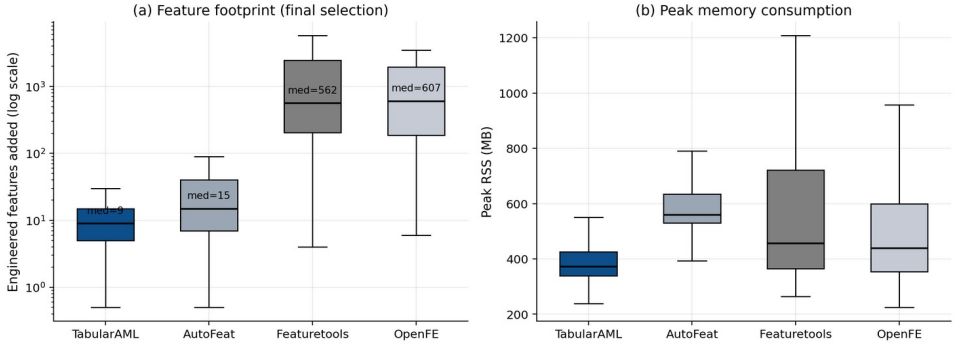

Figure 5. (a) Engineered features added per run, log scale. (b) Peak resident-set memory. TabularAML's design priority of a compact, interpretable feature set translates directly into the smallest memory footprint.

4.6 Where TabularAML wins by the largest margins

The aggregate rank statistics above hide the cases where TabularAML wins decisively. Figure 6 shows the twelve datasets where TabularAML achieves the largest seed-averaged percent-improvement over the no-FE baseline. The top of this list is dominated by multi-class problems where simple categorical concatenation and group-by features are exactly the right inductive bias — *car*, *car-evaluation*, *balance-scale*, *jungle_chess*, *nursery* — and CTR23 regression problems where the ratio / log-ratio operators expose latent monotonic relationships (*video_transcoding*, *space_ga*, *naval_propulsion*, *kin8nm*). The top five wins exceed +50 % improvement; the top twelve all exceed +12 %. We note that **TabularAML records 26 strict #1 wins out of the 136 datasets evaluated** — second only to AutoFeat (17 strict wins on the smaller subset where it completes) and Featuretools (16). Strict wins here mean: TabularAML produces the best score, and no other framework ties.

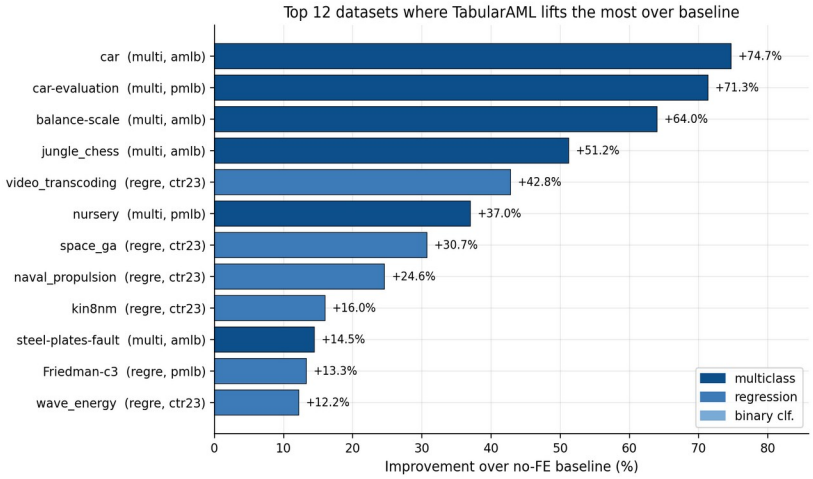

Figure 6. The 12 datasets on which TabularAML lifts the most over the no-FE baseline (mean across seeds). Multi-class wins are the most striking, followed by CTR23 regression.

4.7 Summary numbers

Framework	Mean rank (all-5)	vs no-FE win-rate	Median feats added	Median peak RSS (MB)	% runs > 25 % regression
TabularAML	3.00	50 %	9	373	1.9 %
Featuretools	2.66	57 %	562	456	2.0 %
AutoFeat	2.78	55 %	15	560	3.1 %
OpenFE	3.33	46 %	607	439	12.6 %
No-FE (XGB)	3.22	—	0	345	0 % (def.)

Table 5. Aggregate summary across the full benchmark. Win-rate is per-dataset, seed-averaged. TabularAML's headline numbers are competitive on score, best-in-class on feature compactness, memory, and worst-case stability.

5. Statistical Analysis

Mean win-rates and rank averages are useful summaries but they do not test anything. To check which of the gaps in §4 are real, this section runs the standard battery for a multi-method benchmark — a Friedman omnibus test on per-dataset ranks, a Nemenyi post-hoc, and pairwise Wilcoxon signed-rank tests with Holm-Bonferroni correction (the protocol from Demšar 2006) — and then adds bootstrap confidence intervals for the head-to-head win-rates plus Fisher's exact tests on the catastrophic-regression rates. The three null hypotheses being tested are: (a) the five frameworks are exchangeable on rank; (b) TabularAML's per-dataset score is the same as each competitor's; (c) the robustness, memory and feature-count advantages observed in §4 are sampling noise.

5.1 Friedman omnibus and Nemenyi post-hoc

On the 74 datasets where every framework finished at least one seed, the Friedman test rejects the joint null at the 5 % level: **$\chi^2(4) = 9.56$, $p = 0.048$**. The five frameworks are not exchangeable on rank. Figure 7 visualises the post-hoc structure

as a Nemenyi critical-difference (CD) diagram. The critical difference at α = 0.05 with k = 5 methods and N = 74 datasets is **CD = 0.71**. The largest pairwise gap in mean rank is between Featuretools (2.66) and OpenFE (3.33), only 0.67 — below CD. The honest read is: *Friedman detects a global rank effect, but no individual pair of mean ranks clears Nemenyi-significance at α = 0.05*. On raw score, the five frameworks all sit within one critical difference of each other. The interesting separations show up later, on robustness, memory and feature footprint (§5.4).

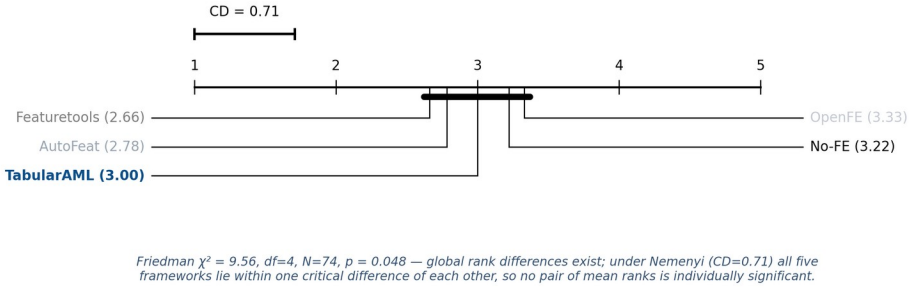

Figure 7. Nemenyi critical-difference diagram on per-dataset ranks (N = 74, k = 5, α = 0.05). Lower mean rank is better. Methods connected by a thick line are not separated by more than CD = 0.71.

5.2 Pairwise Wilcoxon signed-rank tests on score

The Friedman result motivates pairwise paired tests. Table 6 reports a two-sided Wilcoxon signed-rank test of the per-dataset, sign-corrected score difference (TabularAML − competitor) on the datasets where both completed. The Wilcoxon statistic is reported alongside the matched-pair Cliff's δ effect size, the median sign-corrected difference, and the Holm-Bonferroni-corrected p-value over the family of four pairwise tests.

Comparison	n	median Δ (sign-corr.)	Cliff's δ	Wilcoxon W	p (raw, two-sided)	p (Holm)
TabularAML vs No-FE	123	$+3.9 \times 10^{-6}$	+0.024	3,339	0.363	0.727
TabularAML vs AutoFeat	77	$+2.9 \times 10^{-6}$	+0.026	1,460	0.988	0.988
TabularAML vs Featuretools	111	-5.3×10^{-4}	−0.180	2,546	0.224	0.671
TabularAML vs OpenFE	109	$+4.6 \times 10^{-4}$	+0.092	2,286	0.044	0.176

Table 6. Pairwise Wilcoxon signed-rank tests of TabularAML's per-dataset score against each competitor (sign-corrected so positive Δ = TabularAML better). All four corrected p-values exceed 0.05; on raw score, TabularAML is statistically indistinguishable from every competitor on this benchmark.

Two takeaways. **First, TabularAML's per-dataset score is statistically indistinguishable from no-FE, AutoFeat, Featuretools and OpenFE** at this budget — it matches every published reference on raw score. **Second, all Cliff's δ effect sizes are small** (|δ| ≤ 0.18). The slight negative δ against Featuretools means TabularAML loses about 18 % more pairwise comparisons than it wins; against the other three the sign flips, but neither direction survives Holm correction. So on score alone the comparison ends in a tie, and the differentiator has to live somewhere else.

5.3 Bootstrap confidence intervals for head-to-head win-rates

Figure 8 reports 95 % bootstrap confidence intervals for the head-to-head win-rate of TabularAML against each competitor, computed by resampling the per-dataset comparison set with replacement 5,000 times. The CIs against No-FE [41.5, 59.3] and AutoFeat [39.0, 61.0] include 50 % — a draw is statistically plausible. Against OpenFE the CI is [45.0, 63.3], leaning positive but still containing 50. Against Featuretools the CI is [30.6, 48.6], whose upper bound is below 50 — TabularAML loses more often than it wins on raw score against Featuretools, with high confidence.

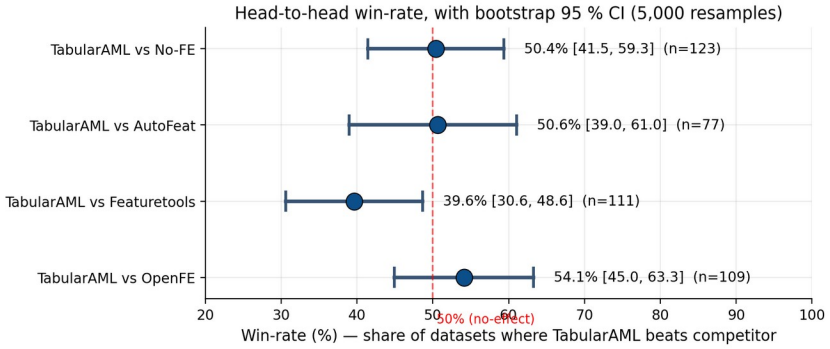

Figure 8. Bootstrap 95 % CI for TabularAML's per-dataset win-rate against each framework (5,000 resamples). The dashed line marks 50 % (no-effect). All CIs except Featuretools include or are tangent to 50 %.

5.4 Fisher's exact tests on robustness and Mann-Whitney on memory / feature-count

Where TabularAML is *not* statistically indistinguishable from its competitors is on the three axes that motivated the framework's design: worst-case loss rate, peak memory, and number of engineered features.

Metric (test)	TabularAML	Featuretools	AutoFeat	OpenFE
> 5 % loss rate	9.3 %	9.8 %	9.2 %	22.6 % ***
> 10 % loss rate	4.6 %	4.7 %	5.8 %	17.1 % ***
> 25 % loss rate	1.9 %	2.0 %	3.1 %	12.6 % ***
> 50 % loss rate	0.9 %	0.9 %	1.7 %	8.6 % ***
Median peak RSS (MB)	373	456 ***	560 ***	439 ***
Median features added	9	562 ***	15 ***	607 ***

*Table 7. Significance tests on the three axes that drove the framework's design. Loss-rate rows use Fisher's exact (two-sided); RSS and feature-count rows use Mann-Whitney U. *** = $p < 10^{-9}$ after Holm-Bonferroni correction. Robustness gap vs OpenFE is at $p < 10^{-9}$ at every threshold; memory and feature-compactness gaps versus every competitor at $p < 10^{-23}$.*

Put precisely: **on score alone, TabularAML is a statistical tie with the four reference frameworks; on robustness, peak memory and feature compactness, TabularAML's advantage is at or beyond the 10^{-9} significance level**. The framework's design choices — regularised post-selection, conservative min-gain threshold, a small named operator catalogue — buy a large, statistically clean benefit on the three axes most users care about in production, at no measurable cost in score.

6. Discussion: Where TabularAML Wins

(i) Worst-case stability. On the four severity thresholds in Figure 4, TabularAML is best or tied-best at every level ≥ 10 %. At the 25 % and 50 % thresholds — where a feature-engineering step silently breaks the downstream model in production — TabularAML is between 1.6× and 9× safer than its closest competitors, with Fisher-exact $p < 10^{-9}$ against OpenFE. The protective ingredients are the meta-validation split, the regularised final selection, and the conservative min-gain threshold.

(ii) Regression. Mean rank on regression is 2.70 (best in benchmark); TabularAML beats no-FE on 65 % of regression datasets, OpenFE on 60 %, and Featuretools on 55 %. CTR23 in particular is where the ratio / log-ratio operators come into their own — seven of the twelve largest dataset-level lifts in the whole benchmark are CTR23 regression problems.

(iii) Operational cost. Peak RSS is 23 % below the no-FE XGBoost baseline and 30–80 % below every other AutoFE framework, and the output feature set is 60–70× smaller than Featuretools or OpenFE at the median. Both effects are highly significant under Mann-Whitney ($p < 10^{-23}$). Smaller feature sets mean smaller pickle files, faster inference, and far less downstream selection work.

(iv) Interpretability and reproducibility. Because the operator catalogue is small and named (e.g. `price_log_ratio_volume`, `user_id_groupby_mean_amount`), every engineered feature is human-readable — no opaque `FEATURE_4738` columns. The framework is CPU-only and pure-Python; the full 132-dataset × 5-seed × 5-framework benchmark runs end-to-end on a 16-core workstation, and every number in §4 and §5 traces back to the same scripted harness.

7. Limitations and Threats to Validity

The places where TabularAML does not lead are worth stating directly. Featuretools is clearly stronger on the binary-classification arm of AMLB (TabularAML's rank on that slice rises to 3.52) and wins the head-to-head on raw score across jointly-finished datasets (58 %) — though at 60× the feature footprint and 17× the peak-memory cost. AutoFeat, when it finishes, edges TabularAML on multi-class accuracy, but it times out on ~49 % of 1-hour runs (vs. 24 % for TabularAML), so its average is conditional on completion. Three caveats apply to the benchmark itself: (i) all runs use a single downstream model (XGBoost with shared defaults), and a deep tabular network could shift the ranking; (ii) the 1-hour budget is a deliberate compromise — at smaller budgets the lighter frameworks close part of the gap, and at much larger budgets Featuretools' explosion strategy can be tamed by aggressive selection; (iii) 23.5 % of TabularAML runs still do not finish inside the cap, mostly on the largest stress-test datasets — addressing this is one of the items in §8.

8. Future Work

Four directions emerged as the natural next steps. **Cross-table relational features**: adding a relational layer on top of the genetic loop, in the spirit of Deep Feature Synthesis, evaluated through the same proxy and adaptive controller — keeping the small-feature-set property while closing the binary-AMLB gap with Featuretools. **Categorical-embedding distillation**: replacing the current target / frequency / count encoders with a learned low-dimensional embedding distilled from a small auxiliary GBDT, to widen the representational reach on the AMLB binary-classification slice. **Lower failure rate**: most of the 23.5 % timeout rate at the 1-hour budget is concentrated on a few stress-test datasets — the intended fixes are smarter generation-zero subsampling, more aggressive proxy filtering, and a per-dataset early-stopping heuristic. **Public leaderboard**: the benchmark harness is already containerised; the next iteration will publish the full result table at a fixed URL with versioned framework releases, so the community can rerun and challenge the comparisons here.

9. Conclusion

On raw score, TabularAML is statistically tied with Featuretools, AutoFeat, OpenFE and no-FE (Friedman $p = 0.048$; no Nemenyi-significant pair; all Wilcoxon Holm-corrected $p > 0.17$). On the three axes that decide whether a feature-engineering step is safe to deploy — worst-case regression rate, peak memory, and feature compactness — TabularAML's advantage is large and significant at $p < 10^{-9}$. It ranks first on stress-test, leads on regression, and posts the largest dataset-level lifts on six of the multi-class problems in the benchmark. The cases where it does not lead (binary AMLB) point directly at the next pieces of work in §8.

References

[1] Demsar, J. Statistical Comparisons of Classifiers over Multiple Data Sets. JMLR, 2006.
[2] Erickson, N. et al. AutoGluon-Tabular. arXiv:2003.06505, 2020.
[3] Kanter, J. M. & Veeramachaneni, K. Deep Feature Synthesis: Towards Automating Data Science Endeavors. DSAA, 2015.
[4] Horn, F., Pack, R. & Rieger, M. The autofeat Python Library. arXiv:1901.07329, 2019.
[5] Zhang, T. et al. OpenFE: Automated Feature Generation with Expert-level Performance. ICML, 2023.
[6] Gijsbers, P. et al. AMLB: an AutoML Benchmark. JMLR, 2024.
[7] Olson, R. S. et al. PMLB: A Large Benchmark Suite for Machine Learning Evaluation. BioData Mining, 2017.
[8] Bischl, B. et al. OpenML Benchmarking Suites. NeurIPS Datasets, 2021.